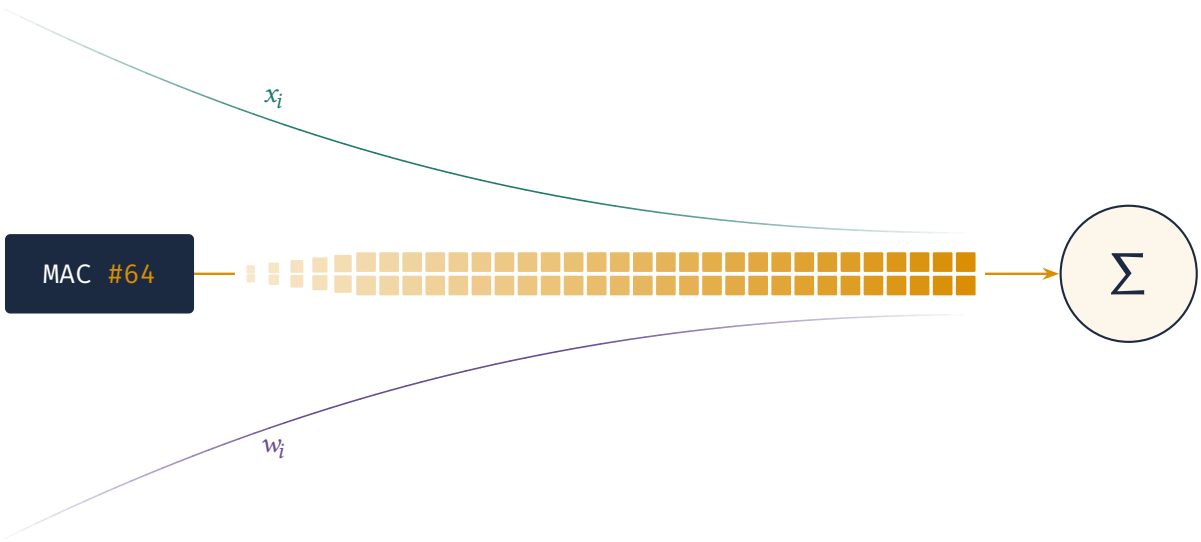


The LS Architecture

A streaming 8-bit CPU for int8 neural inference

Loucas Louka · Stefanos Bordea

Nexys A7-100T (Artix-7 XC7A100T) · VHDL · 9 November 2026

In one paragraph. A neuron is a dot product: 64 multiply-accumulates (MACs) for an 8×8 image. On a classic multicycle CPU every MAC is an instruction, and every instruction pays for fetch, decode, execute and write-back, so the multiplier works one cycle in four. The LS Architecture removes that cost by construction. One instruction, `MAC #n`, names a whole stream of n MACs. After one fetch and one decode the control FSM stays in a single **STREAM** state while the datapath overlaps addressing, multiplying and accumulating across consecutive elements. The elements of a dot product meet only in the accumulator, so the overlap needs no forwarding, no stalls and no hazard logic. The core retires one MAC per clock, the ceiling for one multiplier, from a 12-word program. On one bitstream we will measure a 64-input int8 neuron against the strongest conventional code for the same machine, an unrolled classic MAC: 112 cycles against 300, a predicted 2.68×, read from a cycle counter on the board's display.

Contents

1 The idea **1**

2 A neuron is a dot product **2**

2.1 Why 8-bit integers 2

2.2 How wide the accumulator must be . . . 3

2.3 Requantisation: back to 8 bits 3

3 The LS principle **4**

3.1 Four ways to compute a dot product . . . 4

3.2 What a stream instruction means 5

3.3 The overlap, and why it is safe 5

3.4 Where the speed-up comes from 5

4 Architecture **6**

4.1 Blocks 7

4.2 Memory map 7

4.3 Reads arrive one cycle after their address 7

4.4 Size 8

5 Instruction set **10**

5.1 Instructions 10

5.2 Encoding examples 11

6 Control **12**

6.1 The state machine 12

6.2 The stream contract 13

6.3 One clock, many enables 14

6.4 Run modes and the cycle counter 15

6.5 Build in three levels 16

7 Programs and predictions **16**

7.1 The listings 16

7.2 Predicted counts 17

7.3 How the ratio depends on our choices . . 17

8 On the board **18**

8.1 Inputs and outputs 18

8.2 The demonstration, in three minutes . . 20

8.3 Demo data 20

9 Verification and measurement **20**

9.1 Testbenches 20

9.2 Proving the testbenches work 21

9.3 Measurement plan 21

10 Who builds what **22**

10.1 Modules 22

10.2 Rules that keep two people honest 22

10.3 Learning the whole design 23

11 Schedule **23**

11.1 Milestones 24

11.2 If we fall behind 25

11.3 Risks 25

12 Beyond the course **25**

13 Related work and our position **26**

A Glossary **29**

1 The idea

A neural network spends almost all of its time doing one thing: multiplying inputs by weights and adding up the products. One neuron with 64 inputs is 64 multiply-accumulates, ACC ← ACC + $x_i w_i$, followed by one rescaling step. Everything else is bookkeeping.

On a small, ordinary CPU the bookkeeping costs most of the time. Stefanos' plan for this project (`docs/project-overview.md`) already gives the CPU a `MAC` instruction that multiplies, accumulates and advances two pointers in one go. That is the right start, and every serious DSP since the 1980s has one (Eyre and Bier, 2000). But a multicycle CPU still runs every instruction through **FETCH**, **DECODE**, **EXECUTE** and **WRITEBACK**. A `MAC` is four clock cycles, of which the multiplier is busy for one. Half of the machine's time goes into reading and interpreting the same instruction 64 times. Horowitz quantified the same imbalance for energy: in a simple processor, the arithmetic of one instruction is a tiny fraction of what it costs to fetch and control it (Dally et al., 2020; Horowitz, 2014).

> **The LS principle**
>
> **Pay for control once per stream, not once per operation.** One instruction describes a whole stream of identical operations. The control unit decodes it once, then gets out of the way while the datapath runs the stream at one element per clock.

Figure 1 shows the difference on the same machine. Each row is the same 24-cycle window. In the top three rows, programs built from ordinary instructions spend half of their cycles on control; the amber dots mark the cycles in which the multiplier does useful work. In the bottom row, one `MAC #n` instruction keeps the multiplier busy on almost every clock.

The mechanism. `MAC #64` is fetched and decoded once. The control FSM then enters **STREAM**, a state that loops on itself for 64 clocks. In each **STREAM** clock after the first two, three things happen at once, each to a different element: the next pair (x_{i+2}, w_{i+2}) is addressed in the two operand memories, the previous pair (x_{i+1}, w_{i+1}) is multiplied, and the product before that, $x_i w_i$, is added to the accumulator. Element $i + 1$ never needs element i's result: they only meet inside the accumulator, which adds exactly one product per clock. So the overlap needs none of the forwarding paths, stall logic or flushes that a pipelined CPU needs (Section 3). Two drain cycles finish the last products. `MAC #n` takes $n + 4$ cycles.

> **The claim we will measure**
>
> One bitstream, one 64-input int8 neuron (bias, 64 MACs, requantise with ReLU, output), the same result byte from every program:
>
> - **S**, the LS stream program: 112 cycles, 12 instruction words.
> - **U**, the strongest conventional program for the same CPU, 64 unrolled classic MACs: 300 cycles, 75 words.
> - U / S = 2.68×, and the cost of one more input falls from 4 cycles to 1.
>
> Every cycle count in this document is a prediction derived from the program listings and the cycle rules of the spec. It becomes a result only when the cycle counter on the board shows it.

What is new and what is not. Repeating one instruction without refetching it is old: Texas Instruments' TMS320 DSPs could repeat "a single instruction … up to 256 times", and "instructions that are normally multicycle are pipelined when using the repeat feature, and effectively become single-cycle instructions" (Texas Instruments, 1990). The Harris RTX 2010 stack processor had "streamed" MAC in-

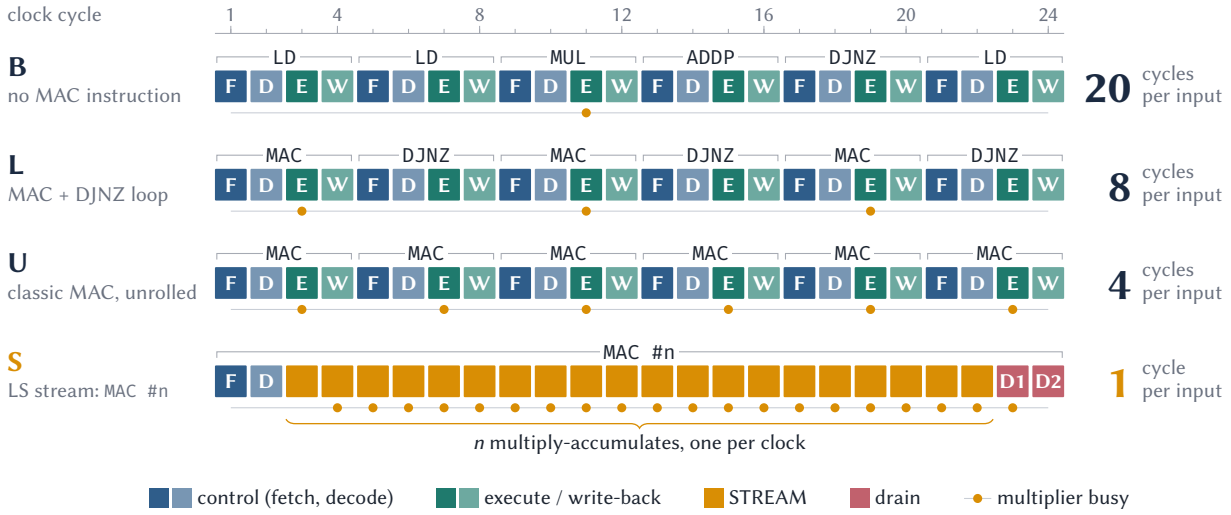


Figure 1. Where the cycles go. The same 24 clock cycles on the same CPU, for four ways of computing a dot product (Section 7). Blue cells are control (fetch, decode); teal cells execute or write back; amber cells are the **STREAM** state; rose cells drain the last products. An amber dot marks a multiply that contributes to the result. B, L and U pay for a fetch and a decode on every element; S pays once.

structions (Harris Semiconductor, n.d.), vector machines have had a vector length register since the Cray-1 (Russell, 1978), and modern RISC-V cores add hardware loops and stream registers (Gautschi et al., 2017; Schuiki et al., 2021). We do not claim the mechanism. Our contribution is a clean form of it on an 8-bit core that two students can build and explain completely, and an honest measurement of what it buys on one bitstream, against the best code the same machine can run without it. Section 13 and the companion state-of-the-art review place the design in its literature.

How to read this document. Sections 2 and 3 explain the arithmetic and the principle. Sections 4 to 6 are the specification the VHDL is written from: datapath, instruction set and control. Section 7 derives every predicted number from real program listings. Sections 8 and 9 cover the board, the demo, the testbenches and the measurements. Sections 10 and 11 split the work between us and plan it against the course dates. Section 12 is where the architecture goes after the course.

2 A neuron is a dot product

Figure 2 is the whole computation. For inputs $x_0, \ldots, x_{63}$, weights $w_0, \ldots, w_{63}$ and a bias b, the neuron computes

$$\mathrm{ACC} = b + \sum_{i=0}^{63} x_i\, w_i, \qquad y = \mathrm{clamp}_{[-128,\,127]}\big(\mathrm{relu}(\mathrm{ACC} \gg sh)\big). \tag{1}$$

A network is many such neurons in layers; the outputs y of one layer are the inputs x of the next. Training, which finds the weights, happens once on a laptop. Inference, which uses them, is Equation (1) over and over, and it is what the LS core runs.

2.1 Why 8-bit integers

Trained weights are real numbers such as 0.7314. Inference works almost as well when every weight and every activation is replaced by an 8-bit integer and a shared scale factor (Jacob et al., 2018). Arm's CMSIS-NN kernels for microcontrollers use the same scheme as ours: int8 values, a wide accumulator, and a power-of-two rescale implemented as an arithmetic shift (Lai et al., 2018). An int8 multiply is

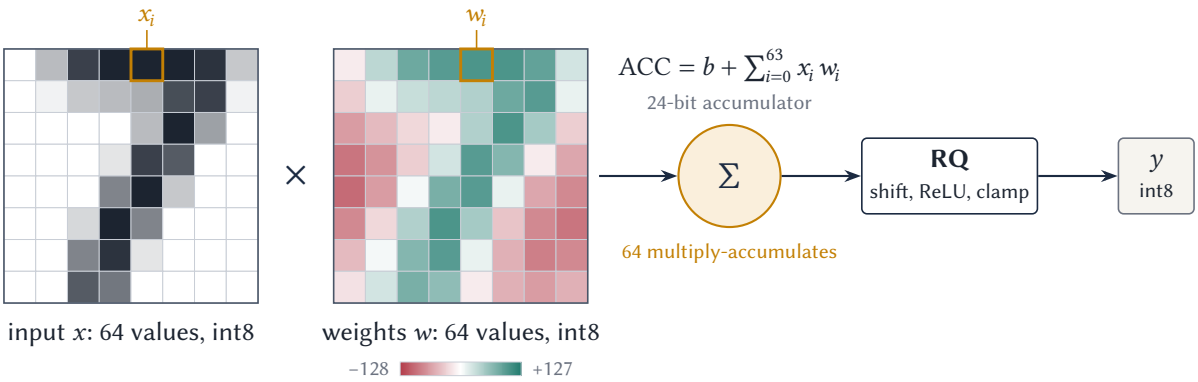


Figure 2. One neuron of an image classifier. The 64 pixels of an 8×8 image and 64 learned weights are multiplied pairwise and summed into a 24-bit accumulator together with a bias b. The requantiser turns the sum back into one int8 value, the input of the next layer.

a single DSP48E1 slice on our FPGA (Xilinx, Inc., 2018), and an int8 value fits our 8-bit registers and memories.

2.2 How wide the accumulator must be

Multiplication doubles the width: two 8-bit signed numbers give a 16-bit signed product. The extreme case is $(-128) \times (-128) = +16{,}384 = 2^{14}$, one more than the largest 15-bit signed value, so the product register `PROD` must be 16 bits and the sign extension to the accumulator must copy bit 15, not bit 14. Summing N products adds $\lceil \log_2 N \rceil$ bits: 64 worst-case products reach 2^{20} and need 22 bits. The accumulator `ACC` has 24 (Figure 3). One `MAC #n` with $n \leq 255$ plus a 16-bit bias cannot overflow it:

$$255 \times 16{,}384 + 32{,}767 = 4{,}210{,}687 \; < \; 2^{23} - 1 = 8{,}388{,}607.$$

Several `MAC #n` instructions chained into one accumulator can overflow; the spec documents that limit rather than detecting it (an overflow flag is an open decision, Section 5).

2.3 Requantisation: back to 8 bits

The next layer expects int8 inputs, so the 24-bit sum must be scaled down. The `RQ` instruction shifts `ACC` right by *sh* bits, keeping the sign (an arithmetic shift, a division by 2^{sh} that rounds towards $-\infty$), optionally applies ReLU (negative values become 0), and saturates the result to $[-128, 127]$ (Figure 4). With *sh* chosen per layer, this is the power-of-two requantisation of CMSIS-NN (Lai et al., 2018). Integer-only inference in general uses a fixed-point multiplier before the shift and a zero point (Jacob et al., 2018); we keep the simpler scheme and list round-to-nearest as an open decision.

> **Signed arithmetic in VHDL**
>
> The labs used `unsigned`; the MAC unit is `signed` from end to end (`ieee.numeric_std`).
>
> - A product is as wide as both operands together ($8 \times 8 \rightarrow 16$). A sum is as wide as its wider operand (`numeric_std` sign-extends the narrower one), so `ACC + PROD` is 24 bits, but the sum of two 16-bit values is 16 bits and can overflow: `resize` first when both operands are narrow.
> - Converting a vector to `signed` only relabels its bits. Sign extension is a separate `resize`.
> - `shift_right` is arithmetic on `signed` and logical on `unsigned`: `RQ` must shift a signed value.
> - -1 in `ACC` reads `FFFFFF`; negating -128 in 8 bits gives -128.

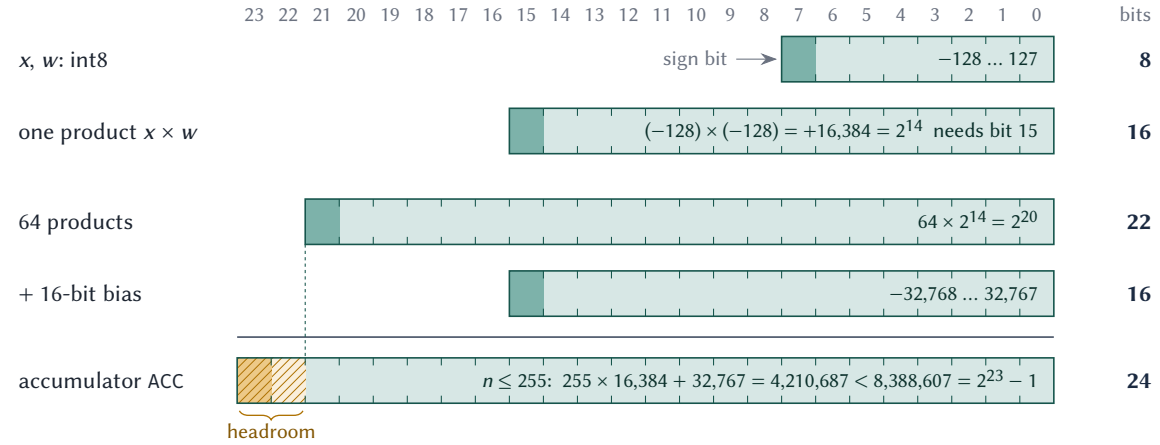


Figure 3. Bit growth through the neuron. A product of two int8 values needs 16 bits because $(-128) \times (-128) = +2^{14}$ does not fit in 15. Sixty-four worst-case products need 22 bits. The 24-bit accumulator leaves headroom for the bias and for any single `MAC #n` up to $n = 255$.

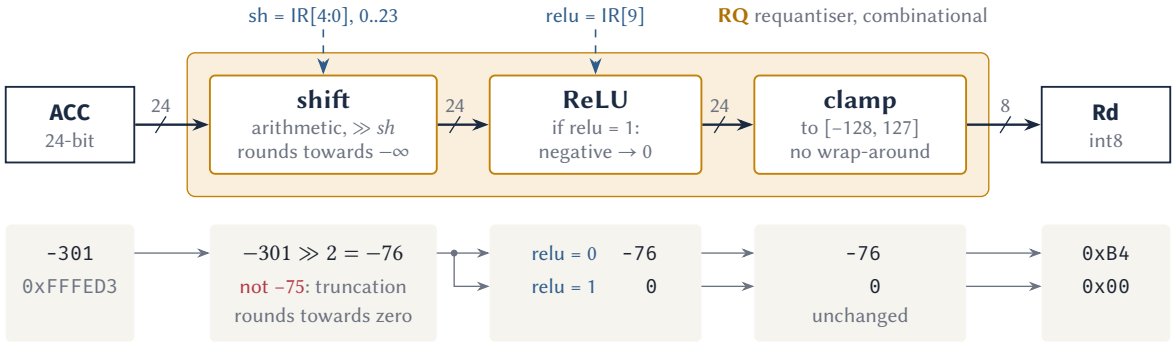


Figure 4. The requantiser, a combinational block in the MAC unit. The arithmetic shift rounds towards minus infinity, which is why $-301 \gg 2 = -76$ and not -75.

3 The LS principle

3.1 Four ways to compute a dot product

The time a program takes is *instructions executed* × *cycles per instruction* × *clock period* (Harris and Harris, 2013; Patterson and Hennessy, 2021). The clock is fixed at 100 MHz, so only the first two factors can move. On one CPU, with one bitstream, there are four ways to write the 64-input neuron, and each removes one kind of overhead from the one before:

	Program	Inner loop	Cycles per input
B	no MAC instruction	`LD`, `LD`, `MUL`, `ADDP`, `DJNZ`: five instructions of 4 cycles	20
L	classic MAC in a loop	`MAC`, `DJNZ`: the MAC fuses load, multiply, add and pointer steps	8
U	classic MAC, unrolled	`MAC` written 64 times: no loop control at all	4
S	LS stream	`MAC #64`: one instruction for the whole stream	1

U is the best a conventional multicycle CPU can do with a MAC instruction: no loop, one instruction per element, four cycles each. It is therefore the fair baseline for the LS claim. B and L are rungs of the ladder that show where the first factors of speed come from (fusing the operation, removing the loop); they are measured, but we never present B/S or L/S as the result.

3.2 What a stream instruction means

X and W are the input and weight memories, addressed by `P0` and `P1`. `MAC #n` with $n \geq 1$ means: *for* $i = 0$ *to* $n-1$, $\mathrm{ACC} \leftarrow \mathrm{ACC} + X[\mathrm{P0}+i] \cdot W[\mathrm{P1}+i]$; *then* $\mathrm{P0} \leftarrow \mathrm{P0} + n$ *and* $\mathrm{P1} \leftarrow \mathrm{P1} + n$. Its architectural effect is exactly that of n classic `MAC` instructions in a row, so a program can switch between U and S without changing any other instruction, and both must produce the same accumulator and the same output byte. Only the timing differs.

3.3 The overlap, and why it is safe

Each element needs three steps, one clock each (Figure 5):

A, address
: Present `P0` and `P1` to the X and W memories; their read registers capture x_i and w_i at the end of the clock. Advance both pointers.

M, multiply
: Multiply the pair waiting in the read registers into `PROD`.

C, accumulate
: Add `PROD`, sign-extended to 24 bits, into `ACC`.

Run one element at a time and `MAC #n` costs $2+3n$ cycles (one fetch, one decode, three steps per element). Overlap the steps of consecutive elements and it costs $n+4$: after two cycles of fill every clock completes one element, and two drain cycles finish the last two.

A pipelined CPU overlaps instructions too, and it pays for that with forwarding paths, hazard detection, stalls and flushes (Harris and Harris, 2013). The stream needs none of these, for three reasons:

- **No data hazards.** Element $i+1$ never reads a result of element i. The only data the elements share is `ACC`, and only step C touches it, adding one product per clock to its own previous value: a one-cycle loop inside one stage. Step A owns the pointers in the same way.
- **No control hazards.** There is no branch inside a stream. The length is known in **DECODE**, so the FSM counts down and leaves **STREAM** at exactly the right clock.
- **No structural hazards.** Each step has its own hardware: the memories and pointers, the multiplier, the adder. Two operand memories (X for inputs, W for weights) deliver both operands of a MAC in the same clock. With one shared memory port the MAC rate could not exceed one every two cycles, whatever the control did (Kung, 1982; Williams et al., 2009).

3.4 Where the speed-up comes from

Figure 6 splits U → S into its two causes. Fetching the instruction once but still running elements one at a time ($2+3n$ cycles for the stream) takes the program from 300 to 238 cycles, 1.26×. Overlapping the three steps takes it on to 112, a further 2.13×. Not refetching is what makes the overlap possible; the overlap is what saves the time.

Two more numbers describe the stream itself. Its *ceiling* is one MAC per clock, because the core has one multiplier: `MAC #64` does 64 MACs in 68 cycles, 94 % of the ceiling, and the whole S program keeps the multiplier busy 57 % of the time. Its *half-performance length*, the stream length at which it reaches half of its peak rate, is $n_{1/2} = 4$: since $n/(n+4) = 1/2$ at $n = 4$, even short streams pay off.

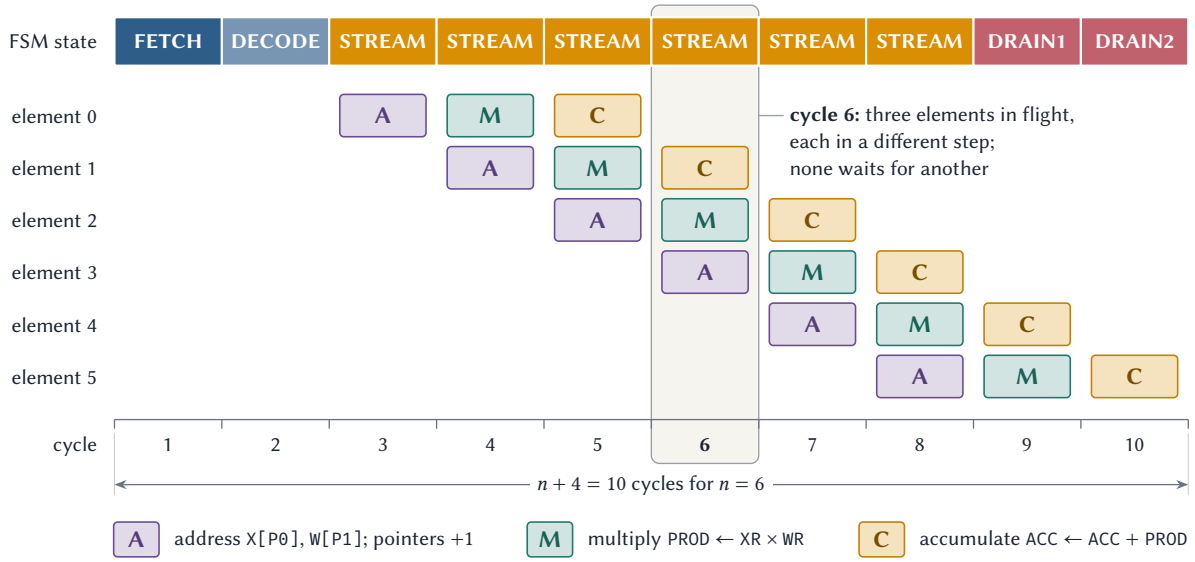


Figure 5. `MAC #6` in space and time. Each element passes through three steps on consecutive clocks: A addresses both operand memories and advances the pointers, M multiplies the pair, C adds the product to ACC. After two cycles of fill, every **STREAM** clock has three different elements in flight, one per step, so the stream retires one MAC per clock.

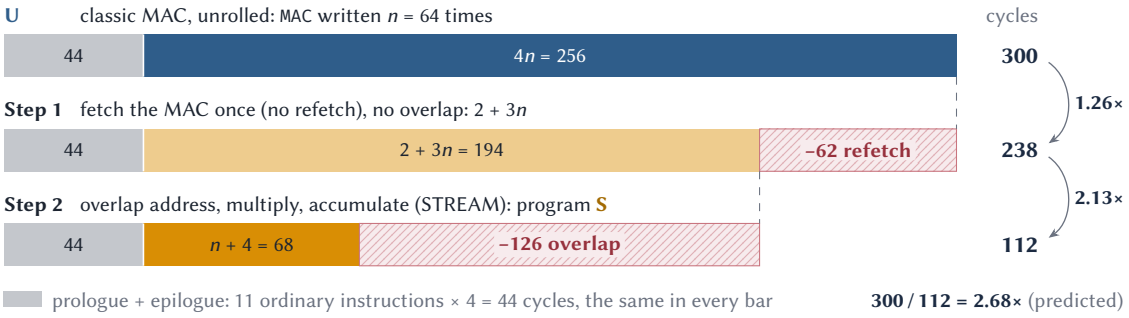


Figure 6. Decomposing the predicted speed-up of S over U. Fetching the MAC once instead of 64 times saves little on its own; overlapping the three steps saves most. Both programs share the same 44 cycles of set-up and output.

> **Why not simply pipeline the CPU?**
>
> A five-stage pipelined CPU with a single-cycle MAC would also approach one MAC per clock, but only on program U: 75 instruction words, 75 fetches and decodes, and a hazard unit, forwarding paths and branch flushing to keep the pipeline correct. The LS core keeps the simple multicycle FSM that the course teaches, fetches and decodes 12 instructions, and gets the same per-element throughput exactly where the work is. The control cost is paid where it is small, once per stream.

4 Architecture

Figure 7 is the complete datapath. It keeps everything in Stefanos' plan (8-bit data, four registers, 16-bit instructions, 12-bit addresses, two auto-advancing pointers, a 24-bit MAC accumulator, a fetch–decode–execute–write-back FSM) and changes four things: separate memories for inputs and weights, registered memory reads, valid bits that let the MAC unit run as a three-step pipeline, and three new FSM states for streams.

4.1 Blocks

Block	Size	Role
PC, IR	12, 16 bits	Program counter and instruction register; IR loads at the end of **FETCH**.
Program ROM	256 × 16	LUT ROM with asynchronous read. An 8-entry start-address table, indexed by SW15..13 at reset, selects one of eight programs.
FSM, decoder	9 states	Section 6. CNT (8 bits) counts the stream down.
R0–R3	4 × 8	General registers. R3 is also the loop counter of DJNZ.
ALU, T, Z, N	8 bits	Add, subtract, logic, move, shifts by one; result register T; zero and negative flags.
P0, P1	2 × 12	Pointers into X and W, each with a +1 incrementer.
X memory	256 × 8	Inputs: four stored 8×8 images of 64 bytes. Address P0[7:0]. Read register XR.
W memory	256 × 8	Weights from address 0. Address P1[7:0]. Read register WR.
Multiplier, PROD	8 × 8 → 16	Signed, one DSP48E1 slice; PROD is registered. An operand mux selects (XR, WR) or two registers.
V1, V2	1 + 1	Valid bits: V1 = the read registers hold a stream pair; V2 = PROD holds a stream product.
Adder, ACC	24 bits	ACC + sext_{24}(PROD), or a 16-bit bias loaded by LDA.
Requantiser	24 → 8	Combinational: arithmetic shift, optional ReLU, saturation (Figure 4).
I/O		IN reads SW7..0; OUT loads OUTR, which drives LED7..0; a view mux feeds the 8-digit display; a 32-bit cycle counter.

4.2 Memory map

Memory	Addressed by	Contents	Written by
Program ROM	PC[7:0]	8 programs, about 200 of 256 words used	VHDL constant
X (inputs)	P0[7:0]	image k at 0x00, 0x40, 0x80, 0xC0 (k = SW7..6)	VHDL constant (ST optional)
W (weights)	P1[7:0]	$w_0 \ldots w_{63}$ at 0x00–0x3F	VHDL constant

All three memories are small enough for LUTs. Block RAM is an optional experiment after Lab 12; it would add a second read cycle if its output register were enabled, and every cycle count would have to be predicted again.

4.3 Reads arrive one cycle after their address

The read registers XR and WR load X[P0] and W[P1] on every enabled clock. Data therefore appears in the cycle *after* its address, which is how FPGA block RAMs behave and what lets the multiplier start a new product every clock without a long combinational path from pointer to multiplier to adder (Figure 8). For ordinary instructions the latency is invisible: the pointers only change at the end of **WRITEBACK**, so by **EXECUTE** the read registers have long held the right value. Inside a stream it accounts for one of the two fill cycles.

> **A classic mistake**
>
> The lecture's RAM example (Code Example 5.7) reads asynchronously. Copied without the read register, data lands in the same cycle as its address and every timing table in this document shifts by one. Add the register, and give it the clock enable (Section 6).

4.4 Size

A planning estimate, not a result: 500 to 800 LUTs, 250 to 350 flip-flops, one DSP48E1 and no block RAM, on a device with 63,400 LUTs, 126,800 flip-flops and 240 DSP slices (Xilinx, Inc., 2020). The stream logic itself (`CNT`, `V1`, `V2`, three states and their enables) should cost about ten flip-flops and a few tens of LUTs. A second build with streaming switched off in the decoder will measure that cost exactly (Section 9).

figure datapath pending

Figure 7. The LS datapath. Blue blocks are control, teal blocks registers and ALU, violet blocks memory, amber blocks the MAC unit, grey blocks board I/O. Numbers on the buses are widths in bits. Dashed blue lines are control signals (register enables and multiplexer selects), all registered or decoded from registers; every register enable is gated by the clock enable `ce`.

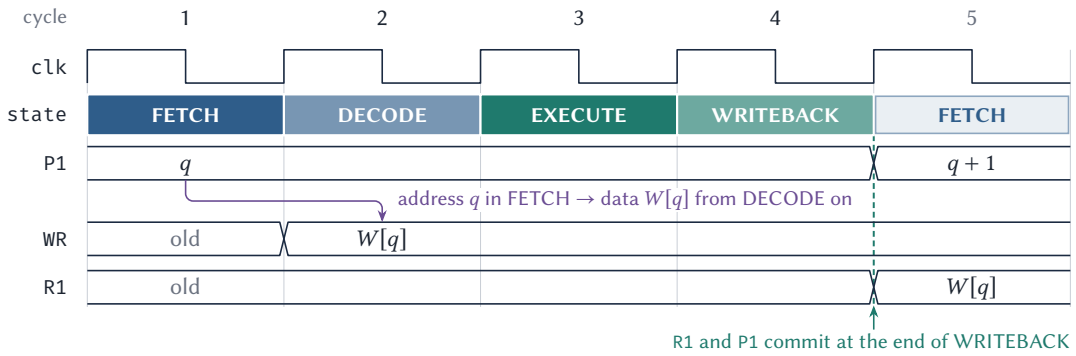


Figure 8. `LD R1, [P1]+`. The address is on `P1` from the first cycle; the data is in the read register `WR` from the second. `R1` and `P1` change together at the end of **WRITEBACK**.

5 Instruction set

Every instruction is 16 bits with a 4-bit opcode in bits 15..12, and all 16 opcodes are used. Six formats share that opcode field (Figure 9); Figure 10 maps the opcodes.

5.1 Instructions

R0–R3, flags, ACC, pointers, OUTR and taken branches change only at the end of **WRITEBACK**, except during a stream. EX and WB below mean the closing clock edge of **EXECUTE** and **WRITEBACK**.

Op	Instruction	Operation	Cycles
0	NOP, HALT, IN Rd, OUT Rs	IN: Rd ← SW7..0; OUT: OUTR ← Rs; HALT: go to **HALTED**	4
1	LDI Rd, imm8	Rd ← imm8	4
2	ADD SUB AND OR XOR MOV SHL SHR SAR	EX: T ← Rd *op* Rs; WB: Rd ← T, Z ← (T = 0), N ← T[7]. Shifts move Rd by one bit.	4
3	MVP Pp, Ra, Rb	Pp ← Ra[3 : 0] & Rb (a 12-bit address from two registers)	4
4	LD Rd, [Pp]{+}	WB: Rd ← XR (for P0) or WR (for P1); if +, Pp ← Pp + 1	4
5	ST Rs, [P0]{+} (optional)	WB: X[P0] ← Rs; if +, P0 ← P0 + 1 (X becomes a RAM)	4
6	MUL Ra, Rb	EX: PROD ← Ra × Rb, signed	4
7	ADDP / LDA Ra, Rb	WB: ACC ← ACC + sext_{24}(PROD) / ACC ← sext_{24}(Ra & Rb)	4
8	MAC ($n = 0$)	EX: PROD ← XR × WR; WB: ACC ← ACC + sext_{24}(PROD), P0 ← P0 + 1, P1 ← P1 + 1	4
8	MAC #n ($1 \le n \le 255$)	n times, one per clock: ACC ← ACC + X[P0] · W[P1], P0 ← P0 + 1, P1 ← P1 + 1	$n + 4$
9	RQ Rd, sh, relu	EX: t = ACC ≫ sh; if relu and $t < 0$ then $t = 0$; T ← clamp(t, −128, 127); WB: Rd ← T	4
A, B	SETP0, SETP1 imm12	P0 or P1 ← imm12	4
C	JMP imm12	PC ← imm12	4
D, E	BZ, BNZ imm12	if Z = 1 (BZ) or Z = 0 (BNZ): PC ← imm12	4
F	DJNZ imm12	EX: T ← R3 − 1; WB: R3 ← T; if T ≠ 0: PC ← imm12; flags unchanged	4

Here & joins bit fields, sext_{24} copies the sign bit up to 24 bits, and ≫ is an arithmetic right shift that rounds down ($-1 \gg 1 = -1$). Function codes: SYS fn = 0 NOP, 1 HALT, 2 IN, 3 OUT; ALU fn = 0 ADD, 1 SUB, 2 AND, 3 OR, 4 XOR, 5 MOV, 6 SHL, 7 SHR, 8 SAR; MVP fn = 0 P0, 1 P1; opcode 7 fn = 0 ADDP, 1 LDA.

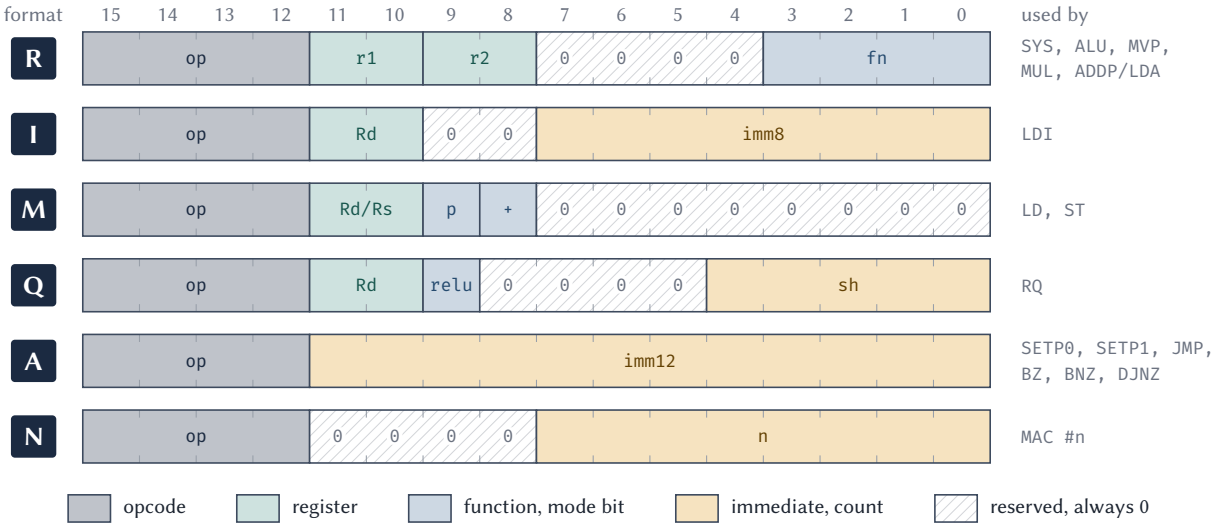


Figure 9. The six instruction formats. Register fields are 2 bits (`R0`–`R3`); `p` selects pointer `P0` or `P1`; `+` requests a post-increment; `sh` is the requantiser shift; `n` is the stream length of `MAC #n`.

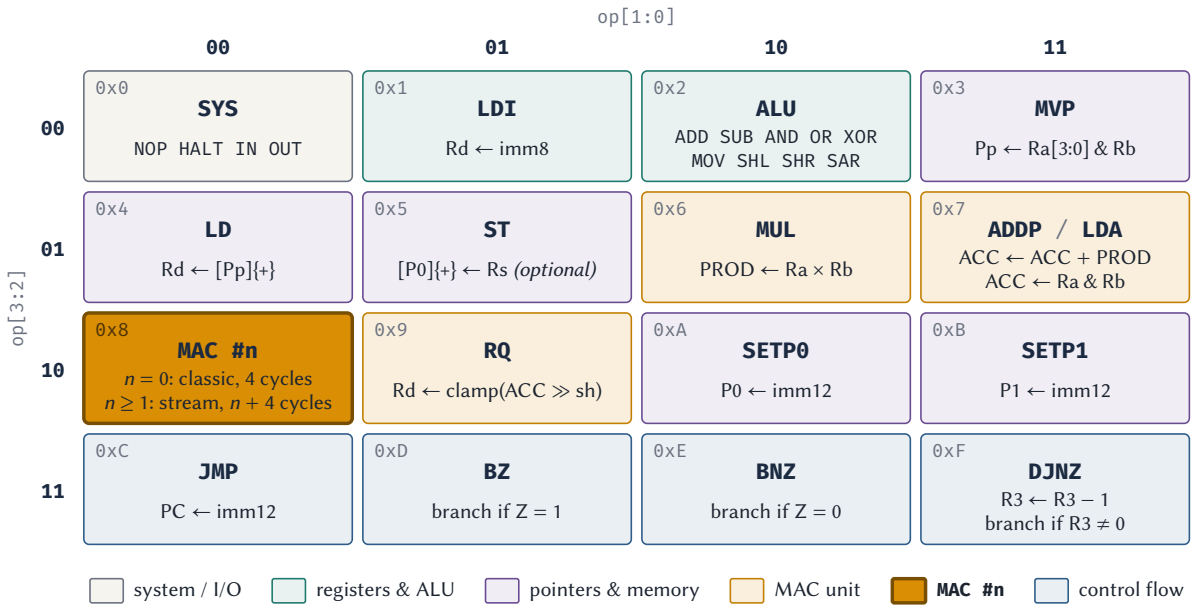


Figure 10. The opcode map. `MAC #n` at `0x8` is the stream instruction: $n = 0$ is the classic single MAC, $n \geq 1$ streams.

5.2 Encoding examples

Instruction	Format	Binary	Hex
`MAC #64`	N	`1000 0000 0100 0000`	`8040`
`MAC`	N	`1000 0000 0000 0000`	`8000`
`LDI R1, 0xC0`	I	`0001 0100 1100 0000`	`14C0`
`AND R0, R1`	R	`0010 0001 0000 0010`	`2102`
`MVP P0, R1, R0`	R	`0011 0100 0000 0000`	`3400`
`LD R1, [P1]+`	M	`0100 0111 0000 0000`	`4700`
`LDA R2, R3`	R	`0111 1011 0000 0001`	`7B01`
`RQ R0, 2, relu`	Q	`1001 0010 0000 0010`	`9202`
`DJNZ 0x009`	A	`1111 0000 0000 1001`	`F009`

Check the spec without an answer key

Each of us encodes the same five instructions from Figure 9, then the other decodes them blind. Every disagreement is an ambiguity in the spec, and it is cheaper to find it on paper than in a waveform.

Open decisions (spec v0.2)

- **RQ rounding**: round down (now) or round to nearest (add 2^{sh-1} before shifting)? The two neuron references must match whichever we choose.
- **Overflow**: a sticky overflow flag on `ACC`, or the documented limit of Section 2?
- **`ST`**: needed for multi-layer networks (Section 12), not for the course demo.
- **Branch on N**: an argmax over ten outputs wants `BN`; there is no free opcode, so two instructions would have to share one.
- **`MAC #1`**: streams in 5 cycles against 4 for `MAC`. Decode it as classic, or simply never write it.

6 Control

6.1 The state machine

The FSM has nine states (Figure 11): the four instruction states of Stefanos' plan, **IDLE** and **HALTED** for the run modes, and **STREAM**, **DRAIN1** and **DRAIN2** for streams.

State	Actions (at the closing clock edge)	Next state
IDLE	none; not counted. During reset `PC` loads the start address of the selected program.	**FETCH** on *go*
FETCH	IR ← PROG[PC]; PC ← PC + 1	**DECODE**
DECODE	decode `IR`; for `MAC #n` with $n \geq 1$: CNT ← n	**STREAM** for `MAC #n` with $n \geq 1$, else **EXECUTE**
EXECUTE	ALU result into `T`; `MUL` or `MAC` product into `PROD`; `RQ` into `T`; `DJNZ`: T ← R3 − 1	**WRITEBACK**
WRITEBACK	the instruction's commits (Section 5)	**HALTED** on `HALT`; **IDLE** in INSTR-STEP; else **FETCH**
STREAM	P0 ← P0 + 1; P1 ← P1 + 1; CNT ← CNT − 1	stay while CNT ≠ 1, then **DRAIN1**
DRAIN1, **DRAIN2**	nothing new is issued; the last pair is multiplied, then added	**DRAIN2**; then **FETCH** (**IDLE** in INSTR-STEP)
HALTED	frozen, counters too	reset only

Every instruction takes four cycles, branches taken or not and `HALT` included; `MAC #n` takes $n + 4$. This uniform timing makes the cycle counts of our programs independent of their data, which is what lets us predict counts exactly and check them on the board.

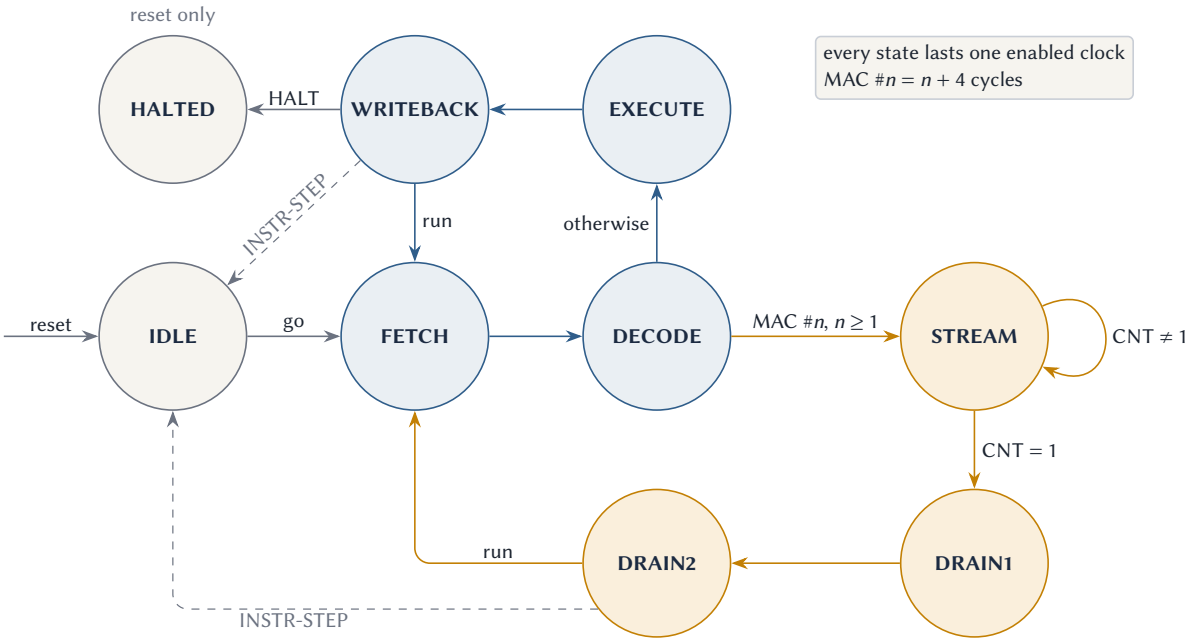


Figure 11. The control FSM. Ordinary instructions take the blue loop, four states of one clock each. `MAC #n` with $n \geq 1$ leaves **DECODE** for **STREAM**, which repeats until the counter reaches 1, then drains for two clocks. Dashed edges are taken only in INSTR-STEP mode.

6.2 The stream contract

The two halves of the core meet in **STREAM**: the control half sequences it, the datapath half computes it. This is the exact interface between them.

The STREAM contract

- V1 ← (state = **STREAM**) and V2 ← V1, on every enabled clock.
- `PROD` loads XR × WR when V1 = 1 (and in **EXECUTE** of `MAC`; `MUL` loads Ra × Rb).
- `ACC` adds sext_{24}(PROD) when V2 = 1 (and in **WRITEBACK** of `ADDP` or `MAC`).
- The multiplier's operand select follows `V1`, *not* "state = **STREAM**", so the last pair is still multiplied in **DRAIN1**.
- `CNT` is loaded in **DECODE**; **STREAM** exits when CNT = 1; both pointers advance on every **STREAM** clock.
- $n = 0$ decodes as the classic `MAC`.

Figure 12 is the contract in action for $n = 4$, starting from ACC = b and P0 = p:

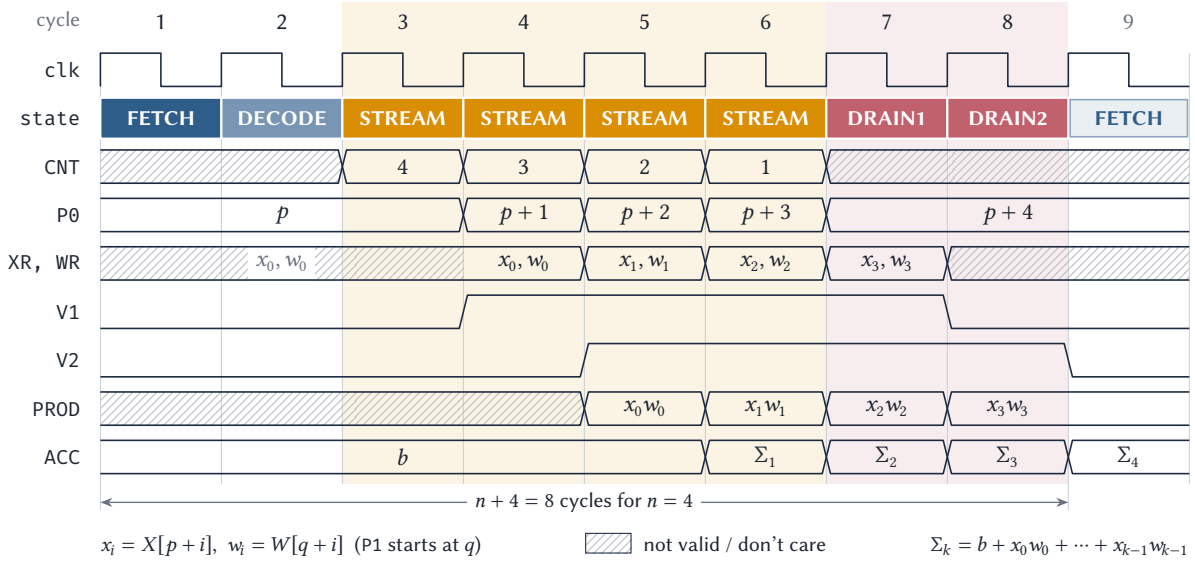


Figure 12. `MAC #4`, cycle by cycle. Values are those held *during* each cycle, which is what the board shows while cycle-stepping. The read registers already hold x_0, w_0 in cycles 1–3, but `V1` is 0 there, so nothing is multiplied until cycle 4. `ACC` lags the pointers by two clocks and holds the full sum when the next instruction is fetched.

Cycle	State	CNT	P0	XR, WR	V1	V2	PROD	ACC
1	**FETCH**	–	p	(not valid)	0	0	–	b
2	**DECODE**	–	p	(not valid)	0	0	–	b
3	**STREAM**	4	p	(not valid)	0	0	–	b
4	**STREAM**	3	$p+1$	x_0, w_0	1	0	–	b
5	**STREAM**	2	$p+2$	x_1, w_1	1	1	x_0w_0	b
6	**STREAM**	1	$p+3$	x_2, w_2	1	1	x_1w_1	$b + x_0w_0$
7	**DRAIN1**	–	$p+4$	x_3, w_3	1	1	x_2w_2	$b + x_0w_0 + x_1w_1$
8	**DRAIN2**	–	$p+4$	–	0	1	x_3w_3	$b + \cdots + x_2w_2$
9	**FETCH**	–	$p+4$	–	0	0	x_3w_3	$b + \cdots + x_3w_3$

The table was checked on a cycle-accurate model of this spec with $b = 0$, $x = 1, 2, 3, 4$ and $w = 5, 6, 7, 8$: `PROD` takes the values 5, 12, 21, 32 and `ACC` ends at 70. Both pointers advance by 4, exactly as after four classic `MAC`s. For the defence, derive the $n = 1$ and $n = 2$ tables from the contract yourself: for $n = 1$ the only product is formed in **DRAIN1**, which is why the operand select must follow `V1`.

6.3 One clock, many enables

CYCLE-STEP needs the CPU to advance one state per button press, yet the design must have exactly one clock. The rule is: *one 100 MHz clock everywhere;* `ce` *is an enable, never a divided or button-driven clock* (Figure 13).

Gated by `ce`	Always running
state register, `PC`, `IR`, `R0`–`R3`, `T`, `Z`, `N`, `P0`, `P1`, `CNT`, `XR`, `WR`, `V1`, `V2`, `PROD`, `ACC`, `OUTR`, cycle and instruction counters, X writes (if `ST` exists)	synchronisers, debouncer and one-pulse; the slow tick; the logic that chooses `ce`; display scanner, digit divider and view mux

Gating the input chain would deadlock CYCLE-STEP, where `ce` itself comes from the button; gating the

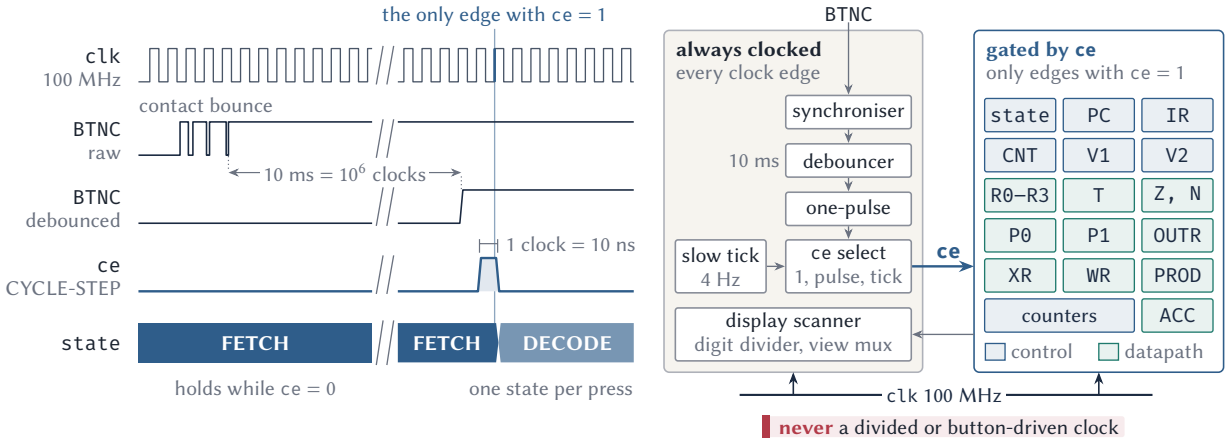


Figure 13. The whole design runs on the 100 MHz board clock. CYCLE-STEP and SLOW do not slow the clock down; they pulse the clock enable `ce`, and every CPU register loads only on clocks where ce = 1. The input chain and the display keep running on every clock.

scanner would freeze the display. Forgetting to gate a register is subtler:

> **The bug that only appears when stepping**
>
> If the read registers ignore `ce`, nothing goes wrong in RUN or INSTR-STEP, where ce = 1 on every clock. In CYCLE-STEP or SLOW, a pause inside **STREAM** lets `XR` and `WR` reload from pointers that have already moved on: one pair is lost and another is counted twice. On the cycle-accurate model with random pauses, ungated read registers gave a wrong `ACC` for program S in 50 of 50 runs and for program U in 0 of 50, because a classic `MAC` moves its pointers only in **WRITEBACK**. Testbench `tb_ce_pause` must catch this bug.

6.4 Run modes and the cycle counter

Two switches select the run mode; one press of BTNC acts on it.

Mode (SW12..11)	`ce`	One press of BTNC
RUN	1	leaves **IDLE** and runs to **HALTED**
INSTR-STEP	1	runs exactly one instruction (`MAC #64` included), waits in **IDLE**, never restarts by itself
CYCLE-STEP	the press pulse	first press: **IDLE** → **FETCH**; then one state per press
SLOW (optional)	a 4 Hz tick	starts the program; then one state per tick

Change the mode switches only in **IDLE** or **HALTED**, or reset afterwards. The 32-bit cycle counter adds one on every enabled clock outside **IDLE** and **HALTED**, so RUN, INSTR-STEP and CYCLE-STEP give the same count for the same program: a free self-check of the control logic. It is cleared by reset, frozen in **HALTED**, and shown on the display.

> **Moore or Mealy? (a defence question)**
>
> In the terms of the course text (Harris and Harris, 2013), a Moore machine's outputs depend on its state only, a Mealy machine's on its state and inputs. The LS controller's outputs in **EXECUTE** and **WRITEBACK** depend on the state *and* on the opcode in `IR`, so if `IR` is an input, the controller is a

> Mealy machine. Every one of its inputs (`IR`, `CNT`, `Z`, `T`, `V1`, `V2`) is itself a register, so its outputs change only after clock edges and cannot glitch from a button. The alternative, used by the multicycle MIPS controller in the course text, is to split **EXECUTE** and **WRITEBACK** into one state per instruction class and make the controller a pure Moore machine. Pick one view and be ready to defend it.

6.5 Build in three levels

The stream is built in three levels, each of which works on its own:

1. **Classic `MAC` only.** The baseline machine: programs U, L and B run and are measured.
2. **Repeat without overlap.** `MAC #n` loops address, multiply and accumulate states without refetching: $2 + 3n$ cycles, S = 238.
3. **Overlapped STREAM.** The contract above: $n + 4$ cycles, S = 112.

Level 2 is a safe fallback if level 3 is late; the go/no-go date is Wednesday 2 December (Section 11).

7 Programs and predictions

Every predicted number in this document comes from the four programs below and the cycle rules of Section 6. They were assembled and run on a cycle-accurate model of the spec, with all four images, random data, worst-case operands and random clock-enable pauses; every run produced the exact integer dot product and the same output byte as a plain reference.

7.1 The listings

All four programs share a prologue that selects the image, points `P0` at it and `P1` at the weights, and loads the 16-bit bias into `ACC`, and an epilogue that requantises, outputs and halts:

```
        IN    R0            ; R0 ← SW7..0 (SW7..6 = image number)
        LDI   R1, 0xC0
        AND   R0, R1        ; R0 ← image * 64
        MVP   P0, R1, R0    ; P0 ← R1[3:0] & R0 = image * 64
        SETP1 WBASE         ; P1 ← address of w0
        LDI   R2, BHI
        LDI   R3, BLO
        LDA   R2, R3        ; ACC ← sext24(bias)
        ; ---- body: one of S, U, L, B ----
        RQ    R0, SH, RELU  ; R0 ← clamp(relu(ACC >>> SH))
        OUT   R0            ; LED7..0 ← R0
        HALT
```

The bodies:

S, the LS stream:

```
        MAC   #64
```

U, classic MAC unrolled:

```
        MAC                 ; 1
        MAC                 ; 2
        ...                 ; 64 in all
```

L, classic MAC in a loop:

```
        LDI   R3, 64
loop:   MAC
        DJNZ  loop
```

B, no MAC instruction:

```
        LDI   R3, 64
loop:   LD    R0, [P0]+
        LD    R1, [P1]+
        MUL   R0, R1
        ADDP
        DJNZ  loop
```

7.2 Predicted counts

	Program	Words	Instructions run	$N = 64$	$N = 32$	Cycles
S	LS stream	12	12	112	80	$48 + N$
U	classic MAC, unrolled	75	75	300	172	$44 + 4N$
L	classic MAC, loop	14	140	560	304	$48 + 8N$
B	no MAC	17	332	1,328	688	$48 + 20N$

The second length, $N = 32$, turns two measurements into a slope: $(112 - 80)/32 = 1$ cycle per input for S, against 4 for U, 8 for L and 20 for B (Figure 14). The slope is the per-element cost with the fixed set-up removed, so it is the cleanest single number for what the stream changes. U at $N = 32$ is only 43 words, so U's slope is measured too.

> **The headline, and only the headline**
>
> S against U on one bitstream: 112 against 300 cycles, 2.68×; slope 1 against 4 cycles per input. Decomposed: 1.26× from fetching once, 2.13× from the overlap (Figure 6). Ceiling: `MAC #64` reaches 94 % of one MAC per clock, the program 57 %.

7.3 How the ratio depends on our choices

Uniform four-cycle instructions make counts easy to predict and check, but they are a design choice, and the ratio moves if we change it. Predicted S and U for other choices:

Design choice	S	U	U/S
every instruction 4 cycles (this spec)	112	300	2.68
classic `MAC` in 3 cycles, the rest in 4	112	236	2.11
ordinary instructions in 3, classic `MAC` in 4	101	289	2.86
everything in 3 cycles	101	225	2.23
STREAM entered after **EXECUTE** ($n + 5$)	113	300	2.65
registered program ROM (one more **FETCH** cycle)	124	375	3.02
repeat without overlap ($2 + 3n$, level 2)	238	300	1.26

Across reasonable choices the stream stays between about 2.1 and 3.0 times faster than the best conven-

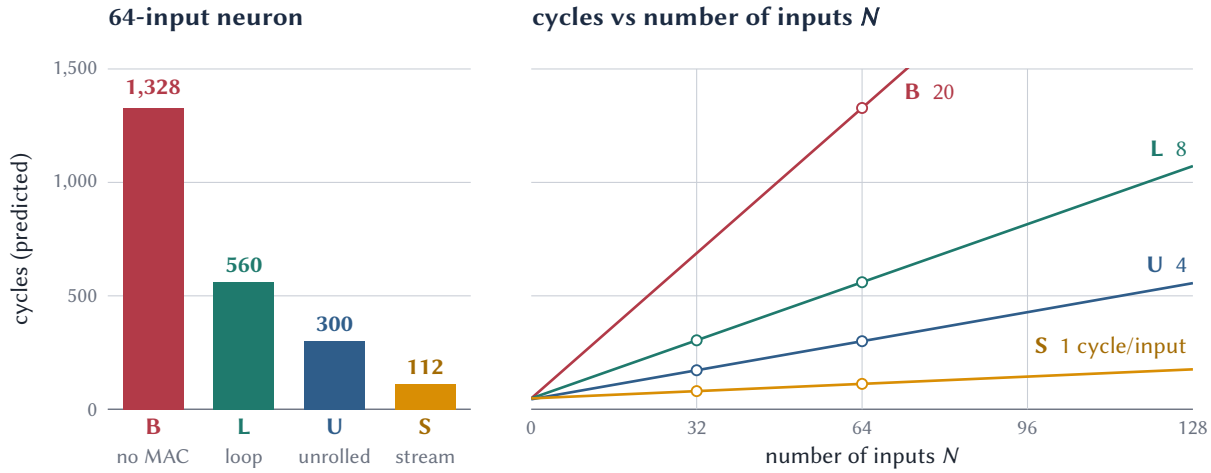


Figure 14. Predicted cycle counts. Left: the 64-input neuron. Right: how each program grows with the number of inputs N; the slopes are the cycles each additional input costs. Marks at $N = 32$ and $N = 64$ are the two lengths we will measure.

tional code, and its slope stays at one cycle per input. If timing closure forces an extra state, we predict again before measuring; we never adjust a measured number to fit a prediction.

What we will not claim

- "fastest", "new" or "first", or any speed-up against a CPU without a hardware multiplier;
- B/S (11.86×) or L/S (5.00×) as the result, or any product of ladder ratios;
- wall-clock time (112 cycles is 1.12 µs at 100 MHz) unless timing is met with worst negative slack WNS ≥ 0;
- energy per program from Vivado's vectorless power estimate, which does not depend on the program;
- any prediction in this document as a result before the board shows it.

8 On the board

8.1 Inputs and outputs

The core uses four of the board's peripherals: slide switches, push buttons, LEDs and the 8-digit seven-segment display (Figure 15). Pins come from Digilent's master constraints file for the board, which must be credited in the report; the table is for cross-checking.

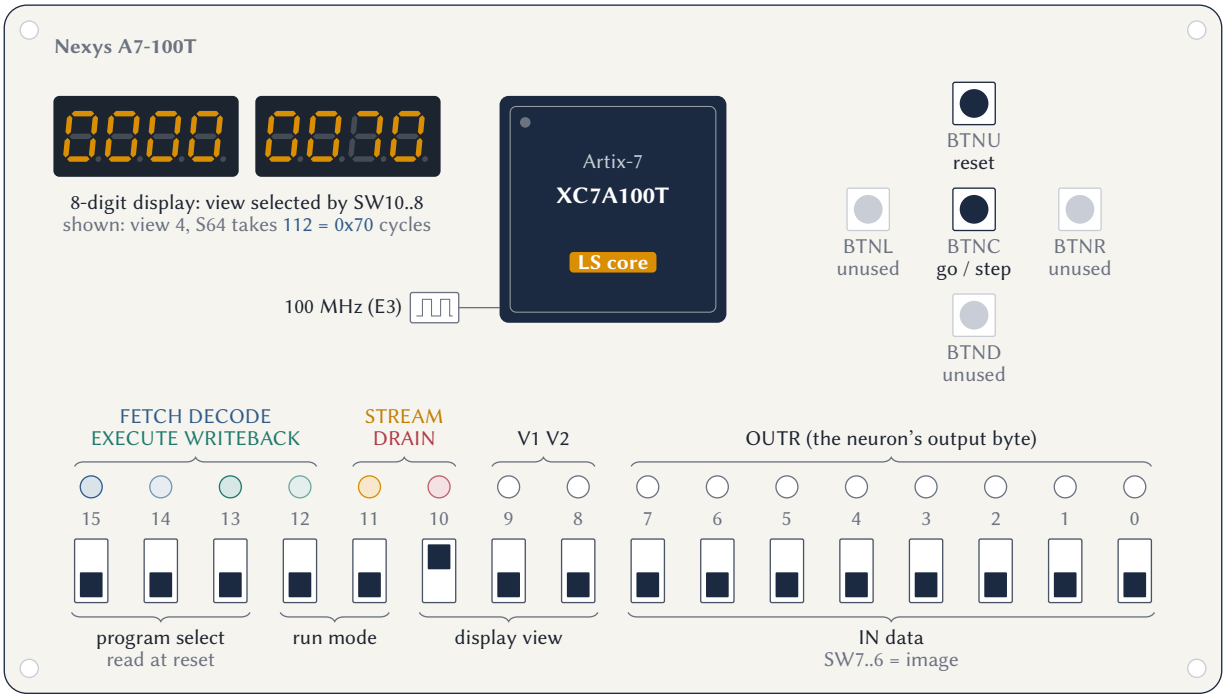


Figure 15. How the LS core uses the Nexys A7-100T. Switches choose the program, the run mode, the display view and the input image; the LEDs show the FSM state, the stream valid bits and the output byte; the display shows any of seven views, here the cycle count of program S (`0x70` = 112).

Port	Pin(s)	Use
`clk`	E3	100 MHz, the only clock; `create_clock` with a 10 ns period
BTNU	M18	synchronous reset, through a two-flip-flop synchroniser
BTNC	N17	go / step: synchroniser, 10 ms debounce, one-clock pulse
SW15..13	V10, U11, U12	program select, read at reset
SW12..11	H6, T13	run mode
SW10..8	R16, U8, T8	display view (SW9 and SW8 are 1.8 V inputs: `LVCMOS18`)
SW7..0	R13, U18, T18, R17, R15, M13, L16, J15	`IN` data; SW7..6 choose the image

LEDs. LED15..12 light in **FETCH**, **DECODE**, **EXECUTE**, **WRITEBACK**; LED11 in **STREAM**; LED10 in either drain state; LED9 and LED8 show `V1` and `V2`; LED7..0 show `OUTR`, the neuron's output byte.

Display. Eight common-anode digits share one set of segment lines. The scanner lights one digit at a time for 1 ms (a tick every 100,000 clocks), so the whole display refreshes every 8 ms, faster than the eye can see; segments and anodes are active-low, and a short blank between digits prevents ghosting. Until the 8-digit driver of Lab 11 exists, one digit or the LEDs are enough.

View (SW10..8)		Program (SW15..13)	
0	PC, state, IR	000	S64
1	R0–R3	001	U64
2	ACC, OUTR	010	L64
3	CNT, P0, P1	011	B64
4	cycle counter (8 hex digits)	100	S32
5	instruction counter	101	U32
6	PROD, V1, V2, Z, N	110	L32
		111	MAC #8 demo

8.2 The demonstration, in three minutes

1. **Watch a stream happen.** Cycle-step the MAC #8 program with views 3 and 2. LED15..12 show **FETCH**, **DECODE**, then LED11 lights for **STREAM**; with each press CNT falls and P0, P1 rise by one; V1 lights one press into the stream and V2 one press later; two drain presses finish it. The board matches Figure 12 press by press, with eight **STREAM** presses instead of four.
2. **Race the four programs.** RUN mode, view 4, reset between runs: the display shows the counts of S64, U64, L64 and B64, and LED7..0 show the same output byte every time.
3. **Change the data, not the time.** Flip SW7..6 to another image: the output byte changes, the cycle counts do not.
4. **The slope.** S32 and L32: thirty-two fewer inputs save 32 cycles with the stream and 256 with the loop.

8.3 Demo data

A hand-made neuron works better on the board than a trained one: a template detector for one digit, with small positive weights on the digit's strokes and small negative weights around them, drawn on squared paper. Four stored images give clearly different sums. The shift *sh* is the smallest that brings the largest sum within 127; the bias and ReLU are set so the four output bytes differ and none is 00 or 7F (an output stuck at the ReLU floor or the clamp limit would hide errors). All four outputs are written on the prediction sheet before the board is switched on. A negative check for the requantiser: ACC $= -301$, $sh = 2$ gives -76 (B4) with ReLU off and 00 with ReLU on.

9 Verification and measurement

9.1 Testbenches

Every testbench is self-checking in the style of the lab templates: it counts mismatches, prints the first few failures, and ends with a clear PASSED or NOT YET. Expected values come from the spec tables or from our own reference models, never from the design under test. Simulation runs in Vivado's XSim in the lab and in nvc on a laptop, so both tools are set to the same VHDL standard.

#	Testbench	What it proves	Written by	When (Section 11)
1	`tb_alu`	all 65,536 operand pairs for every function, flags included	Loucas	by 20 Nov
2	`tb_mul_rq`	all 65,536 signed products ($(-128)\times(-128)$ needs bit 15); `RQ` for $sh = 0..23$ at the 24-bit edges, ReLU on and off, negatives rounding down	Loucas	by 20 Nov
3a	`tb_fsm` (M1)	every M1 path, one clock per state, RUN to **HALTED**, INSTR-STEP parks in **IDLE**, reset from every state	Stefanos	M1
3b	`tb_fsm` (M3)	**STREAM** lasts n clocks; parking after **DRAIN2**; the first CYCLE-STEP press	Stefanos	M3
4	`tb_instr`	a small program per instruction group, checked after each instruction	both	M1–M2
5	`tb_mem`	data at $t+1$; held while ce $= 0$; X and W read together	Loucas	M2
6	`tb_neuron`	S, U, L, B at $N = 64$ and 32 on four images: outputs match both references, counts match the predictions	both	M2–M3
7	`tb_stream`	$n = 1, 2, 3, 4, 64, 255$; $64\times(-128)(-128) = 2^{20}$; random vectors; $n+4$ cycles; pointers end at base $+\,n$	Stefanos	2 Dec
8	`tb_ce_pause`	tests 6 and 7 with `ce` dropped for random 1–20 cycle gaps: same `ACC`, same count	Stefanos	with CYCLE-STEP
9	`tb_io`	a bouncing button gives one pulse; one active-low anode at a time; scanning continues while ce $= 0$	both	M2

9.2 Proving the testbenches work

A testbench that has never failed proves nothing. Before M3 we inject each of these bugs, one at a time, and record which testbench catches it:

Injected bug	Must be caught by
sign extension of `PROD` from bit 14 instead of bit 15	`tb_mul_rq`, `tb_stream`
read registers `XR`, `WR` not gated by `ce`	`tb_ce_pause`
pointer increment moved from **WRITEBACK** into **DECODE**	`tb_instr`, `tb_neuron`
operand select decoded from "state = **STREAM**" instead of `V1`	`tb_stream` ($n = 1, 2$)
STREAM exit at CNT $= 0$ instead of 1	`tb_stream`, `tb_fsm`
logical instead of arithmetic shift in `RQ`	`tb_mul_rq`

9.3 Measurement plan

1. **Freeze predictions first.** Each of us derives every prediction separately; we compare, then freeze them before the first board measurement.
2. **Baselines on the plain machine.** At M2, before the stream is in the bitstream, measure U64, L64 and B64 on the board and record the date and build. The final bitstream must reproduce them exactly.
3. **The final bitstream.** Each program once per image (output byte and count); S64 and U64 three times each; everything recorded on a printed sheet next to the predictions.
4. **Two lengths.** The $N = 32$ programs give the slopes of S, U and L; B has no $N = 32$ program on the board, so its slope stays a prediction checked in simulation. One short program in RUN, INSTR-STEP and CYCLE-STEP must give the same count.

5. **The hardware cost.** Two Vivado builds of one source: normal, and stream-off (the decoder treats every `MAC #n` as `MAC`). For both: utilisation per module (LUT, FF, DSP48E1, LUTRAM), worst negative slack at 100 MHz, and on-chip power. The difference is what the stream costs in hardware.
6. **If the stream works only in simulation**, report its XSim counts labelled "simulation" next to the board-measured U, L and B.

10 Who builds what

We both write VHDL, and we both have to explain every part of the design at the defence. The split below gives each of us one half of the core to design and the other half to verify, so each of us ends up knowing both halves: one from the inside, one from its specification.

10.1 Modules

Module	Designed by	Verified by	Needed for
Control half			
FSM and instruction decoder	Loucas	Stefanos (`tb_fsm`)	M1
`PC`, start-address table, program ROM, branches	Loucas	Stefanos (`tb_instr`)	M1
Run modes and `ce` logic	Loucas	Stefanos (`tb_fsm`, `tb_ce_pause`)	M1
Stream control: **STREAM**, `CNT`, `V1`, `V2`, enables	Loucas	Stefanos (`tb_stream`)	M2–M3
Cycle and instruction counters	Loucas	Stefanos	M1
Display scanner and view mux	Loucas	Stefanos (`tb_io`)	M2
Datapath half			
Register file and write-back mux	Stefanos	Loucas (`tb_instr`)	M1
ALU, `T` and flags	Stefanos	Loucas (`tb_alu`)	M1
Input chain: synchroniser, debouncer, one-pulse	Stefanos	Loucas (`tb_io`)	M1
LEDs, top level and constraints file	Stefanos	Loucas	M1
Pointers, X and W memories, read registers	Stefanos	Loucas (`tb_mem`)	M2
Multiplier, `PROD`, adder, `ACC`	Stefanos	Loucas (`tb_mul_rq`)	M2
Requantiser	Stefanos	Loucas (`tb_mul_rq`)	M2
Together			
Programs, demo data, `tb_neuron`	both	both	M2
Neuron reference model	one each, independently	compared	M2
Report, measurements, demo	both	both	M3

10.2 Rules that keep two people honest

- **Nobody verifies their own module.** The verifier writes the testbench from the spec tables of this document, not from the designer's code. If the spec is ambiguous, the testbench will expose it.
- **One owner for the stream.** The stream control is designed end to end by one person (states, `CNT`, `V1`, `V2`, the `PROD` and `ACC` enables, the pointer increments) and tested by the other (`tb_stream`,

`tb_ce_pause`). The contract of Section 6 is the interface between them.

- **Two references.** Each of us writes an independent neuron reference (a spreadsheet or a short program). Their outputs must agree before they become the expected values of `tb_neuron`.
- **Small commits, reviewed.** Each module goes in through a pull request that the other person reads before merging.
- **A dated log.** Who designed, wrote and verified what, kept as we go. The contribution statement at the end is then a copy, not a reconstruction.

10.3 Learning the whole design

Every week, a 15-minute teach-back: each of us explains the newest block of our half to the other, at the whiteboard, without notes. At the rehearsal before the defence each of us presents the *other's* half. We are ready when both of us can answer these without notes:

1. `LD`, cycle by cycle: when does the data appear, and why?
2. Why does `MAC #n` take $n + 4$ cycles, and why does `ACC` lag the pointers by two?
3. What breaks if a read register ignores `ce`, and in which run modes?
4. Why is `PROD` 16 bits, and how many worst-case products fit in 24 bits?
5. `RQ` of -301 with $sh = 2$ and no ReLU: why -76 and not -75?
6. Is the controller a Moore or a Mealy machine?
7. Why clock enables and not a slower clock?
8. Why two flip-flops before the debouncer?
9. How do eight digits share one set of segment pins?
10. How much of $300 - 112$ comes from not refetching, and how much from the overlap?
11. A pipelined CPU also does one MAC per clock. What does the LS core save?
12. What do WNS, LUT, FF and DSP figures mean, and why is the power estimate the same for every program?
13. What happens at power-up, before the first reset?
14. Which earlier machines repeated a MAC without refetching it, and what is different here?

11 Schedule

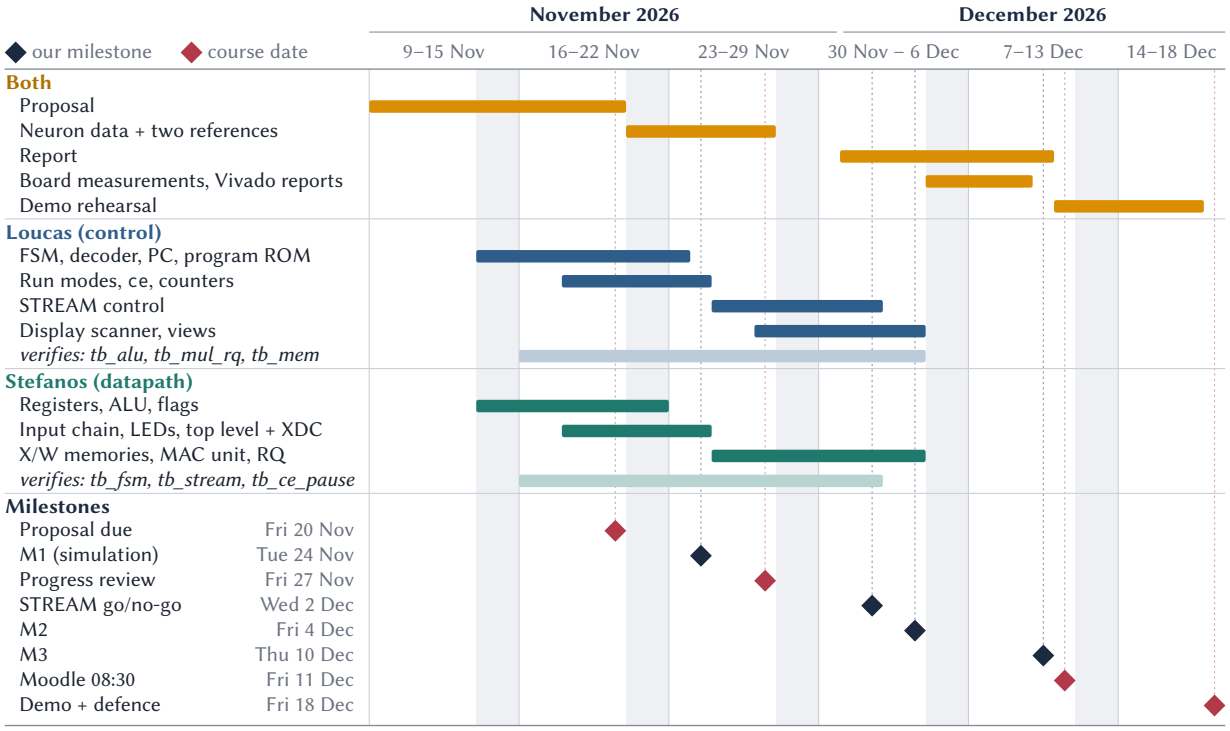


Figure 16. The plan from 9 November to the demonstration. Blue bars are Loucas' modules, teal bars Stefanos', amber bars joint work; lighter bars are the testbenches each of us writes for the other's half. Diamonds are our milestones and the course deadlines.

11.1 Milestones

Date	Milestone	Done when
Fri 20 Nov	Proposal	submitted with the principle, the I/O, three milestones and the work split
Tue 24 Nov	**M1**: a CPU that runs	in simulation: FSM, `PC`, `IR`, program ROM, `R0`–`R3`, `LDI`, ALU, `OUT`, `JMP`, `DJNZ`, `HALT`, RUN and INSTR-STEP; `tb_alu`, `tb_fsm` (M1) and `tb_instr` pass; state on LED15..12
Fri 27 Nov	Progress review	M1 shown, on the board if time allows
Wed 2 Dec	Stream go/no-go	`tb_stream` passes with constant X/W tables; otherwise fall back to level 2
Fri 4 Dec	**M2**: the neuron	memories, MAC unit and `RQ`; L64 and U64 pass `tb_neuron` at their predicted counts; baselines measured on the board
Thu 10 Dec	**M3**: final	stream on the board, all measurements, both Vivado builds, report complete
Fri 11 Dec	Submission	Moodle closes at 08:30
Fri 18 Dec	Demo and defence	three-minute demo (Section 8); both of us answer on the whole design

The labs teach the skills just in time: state machines in Lab 10 (20 Nov), counters and the 8-digit display in Lab 11 (27 Nov), memories, utilisation and power in Lab 12 (4 Dec). Memories are needed before Lab 12, so we read ahead on them in the last week of November. The report is written as we go: specification, ISA, FSM and I/O after M1; datapath, memory and verification after M2. The last week only adds measurements, resources, power and discussion.

11.2 If we fall behind

The course floor is never cut: a working CPU with the classic `MAC`, program L64 on the board, the cycle counter, INSTR-STEP, the LEDs and a display. The stream sits above that floor in levels, and each trigger below costs one thing, not the project.

Trigger	What we cut
27 Nov: M1 not on the board	SLOW mode, the instruction counter, the $N = 32$ programs
2 Dec: `tb_stream` fails	overlapped **STREAM**; keep level 2, repeat without overlap
2 Dec: L64 fails `tb_neuron`	B64 and three of the four images (the image-select prologue stays, so no count changes)
8-digit driver late	show the low 16 bits of the cycle counter on LED15..0 (the largest prediction, 1,328, fits)
7 Dec: board time short	block RAM experiment, `ST`, CYCLE-STEP (`tb_ce_pause` stays while any mode pulses `ce`)

11.3 Risks

Risk	What can go wrong	Mitigation
Schedule	the 20–27 Nov crunch (proposal, Lab Report 3, the CSEN 203 project); limited board time	milestones checkable in simulation; printed test sheets so a board session takes 20 minutes
Read latency	data used in its own address cycle; a registered program ROM; a block RAM output register	"data the cycle after its address"; Figure 8; `tb_mem`; asynchronous program ROM
Clock enables	a divided or button clock; an ungated read register; a gated input chain	one 100 MHz clock; the `ce` table; `tb_ce_pause`
Stream control	exit one cycle early or late; the operand select in **DRAIN1**	`CNT` loaded in **DECODE**, exit at 1; `tb_stream` for $n = 1..4$
Signed arithmetic	unsigned multiply; extension from the wrong bit; logical shift in `RQ`	`signed` throughout; exhaustive `tb_mul_rq`
Board I/O	bounce, metastability, missed presses; ghosting on the display	synchronisers on every switch and button; 10 ms debounce; one-clock pulses; blanking
Vivado	power-up state; memories or the multiplier inferred differently; timing at 100 MHz	reset after programming; read the synthesis log from the first M2 build; if WNS < 0, add a state and predict again

12 Beyond the course

The course core is the first station of a longer line (Figure 17). Stefanos' plan already names the destination: a compiler that turns a small trained model into a program for our CPU, handwritten digit recognition on the board, and a hardware accelerator. CFU Playground walks a similar path with custom int8 instructions on a 32-bit soft CPU (Prakash et al., 2023); the LS principle lets us walk it without giving up the simple 8-bit core.

Toolchain. An assembler and an instruction-set simulator first: labels, encodings, a memory initialisation file for the ROMs, and a cycle prediction for every program. The simulator becomes the golden

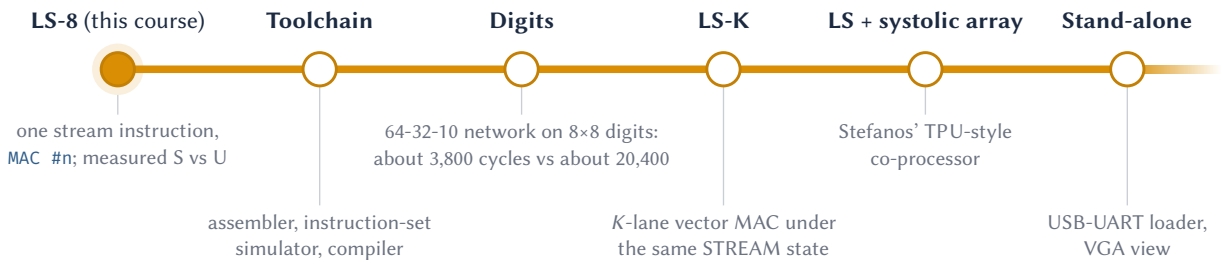


Figure 17. Where the LS Architecture goes after December. Each station is measured against the one before it with the same cycle counter.

model for every later testbench.

Digits. A 64-32-10 network on the 8×8 handwritten digits of the UCI optical digits data (Alpaydin and Kaynak, 1998): 64 inputs, 32 hidden neurons, 10 outputs, 2,368 weights. Each neuron is one `MAC #n` plus about seven instructions of set-up, requantisation and store, so the whole network is about

$$32 \times (28 + 68) + 10 \times (28 + 36) + 120 \approx 3{,}800 \text{ cycles}$$

including an argmax of about 120 cycles: roughly 38 µs per digit at 100 MHz (derived), against about 20,400 cycles with the classic MAC in a loop. The weights need a 4K × 8 W memory, one block RAM; the hidden layer's outputs need `ST`.

Fewer cycles per stream. Overlap the drain of one stream with the fetch of the next instruction; fetch the bias from the weight stream; fuse requantise-and-store. Each shortens the fixed 48 cycles around a stream, which matters once streams are short.

LS-K: more lanes under the same state. The stream state does not care how much work happens per clock. Widen X and W to K bytes and put K multipliers under the same **STREAM** state, and the same 12-word program runs K times faster on long vectors, with the same FSM. The FPGA has 240 DSP slices.

A systolic co-processor. Stefanos' accelerator plan, a small grid of multiply-accumulate cells in the style of the TPU (Jouppi et al., 2017; Kung, 1982), becomes a co-processor that the LS core feeds with streams. A one-cell build of the array, compared with the streamed MAC on the same counter, keeps the comparison honest.

Stand-alone. A USB-UART loader for programs and weights (the FPGA receives on C4 and transmits on D4) and a VGA view of the image and the answer.

13 Related work and our position

The companion document *The LS Architecture: state of the art* surveys the literature in depth. This table is the short version: each idea the LS core uses, where it comes from, and what we do differently.

Idea	Prior art	The LS core
Repeat one instruction without refetching it	TMS320 "repeat feature" with `RPT`/`RPTK` and `MAC`, up to 256 times (Texas Instruments, 1990); RTX 2010 "streamed" MACs (Harris Semiconductor, n.d.)	the count lives inside the instruction (`MAC #n`) instead of a separate repeat prefix
A vector length set per operation	Cray-1 vector length register (Russell, 1978)	a scalar 8-bit core; the stream length is the vector length
Zero-overhead loops and address generators	DSP processors (Eyre and Bier, 2000); RI5CY hardware loops and post-increment loads (Gautschi et al., 2017)	no loop at all inside a stream; two pointers advance in hardware
Streams fed straight to the arithmetic	stream semantic registers (Schuiki et al., 2021)	the operand memories are read directly by the stream, with no register file in between
int8 MAC with a wide accumulator, power-of-two requantisation	CMSIS-NN (Lai et al., 2018); integer-only inference (Jacob et al., 2018); custom int8 instructions on soft CPUs (Prakash et al., 2023)	the same arithmetic on an 8-bit core with a 24-bit accumulator
Many MACs per clock	systolic arrays (Kung, 1982); the TPU (Jouppi et al., 2017)	one MAC per clock now; K lanes and an array are later stations (Section 12)

None of these ideas is ours. What the LS project contributes is a version of the streaming idea small enough to build in five weeks and explain completely; a measurement on one bitstream against the strongest code the same machine can run without it, with the speed-up split into its causes; and a design that grows into more lanes and a co-processor under the same control state. The ambition is to understand one mechanism completely and measure it honestly, then build on it.

References

E. Alpaydin and C. Kaynak. Optical recognition of handwritten digits. UCI Machine Learning Repository, dataset, 1998.

William J. Dally, Yatish Turakhia, and Song Han. Domain-specific hardware accelerators. *Communications of the ACM*, 63(7):48–57, 2020. doi: 10.1145/3361682.

Jennifer Eyre and Jeff Bier. The evolution of DSP processors. *IEEE Signal Processing Magazine*, 17(2): 43–51, 2000. doi: 10.1109/79.826411.

Michael Gautschi, Pasquale Davide Schiavone, Andreas Traber, Igor Loi, Antonio Pullini, Davide Rossi, Eric Flamand, Frank K. Gürkaynak, and Luca Benini. Near-threshold RISC-V core with DSP extensions for scalable IoT endpoint devices. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 25 (10):2700–2713, 2017. doi: 10.1109/TVLSI.2017.2654506.

David Money Harris and Sarah L. Harris. *Digital Design and Computer Architecture*. Morgan Kaufmann, 2nd edition, 2013.

Harris Semiconductor. *RTX 2000 Family Programmer's Reference Manual*, n.d. Section 7.7.1, Streamed MAC instructions on the RTX 2010.

Mark Horowitz. Computing's energy problem (and what we can do about it). In *IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, pages 10–14, 2014. doi: 10.1109/ISSCC.2014.6757323.

Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2704–2713, 2018. doi: 10.1109/CVPR.2018.00286.

Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, et al. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, pages 1–12, 2017. doi: 10.1145/3079856.3080246.

H. T. Kung. Why systolic architectures? *Computer*, 15(1):37–46, 1982. doi: 10.1109/MC.1982.1653825.

Liangzhen Lai, Naveen Suda, and Vikas Chandra. CMSIS-NN: Efficient neural network kernels for Arm Cortex-M CPUs, 2018. arXiv:1801.06601.

David A. Patterson and John L. Hennessy. *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. Morgan Kaufmann, 2nd edition, 2021.

Shvetank Prakash, Tim Callahan, Joseph Bushagour, Colby Banbury, Alan V. Green, Pete Warden, Tim Ansell, and Vijay Janapa Reddi. CFU Playground: Full-stack open-source framework for tiny machine learning (TinyML) acceleration on FPGAs. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 157–167, 2023. doi: 10.1109/ISPASS57527.2023.00024.

Richard M. Russell. The CRAY-1 computer system. *Communications of the ACM*, 21(1):63–72, 1978. doi: 10.1145/359327.359336.

Fabian Schuiki, Florian Zaruba, Torsten Hoefler, and Luca Benini. Stream semantic registers: A lightweight RISC-V ISA extension achieving full compute utilization in single-issue cores. *IEEE Transactions on Computers*, 70(2):212–227, 2021. doi: 10.1109/TC.2020.2987314.

Texas Instruments. *TMS320 Second-Generation Digital Signal Processors*, 1990. Data sheet SPRS010B, May 1987, revised November 1990.

Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

Xilinx, Inc. *7 Series DSP48E1 Slice User Guide*, 2018. UG479, v1.10.

Xilinx, Inc. *7 Series FPGAs Data Sheet: Overview*, 2020. DS180, v2.6.1.

A Glossary

Term	Meaning
`ACC`	the 24-bit signed accumulator of the MAC unit
BRAM	block RAM: the FPGA's dedicated 36 Kb memories, read through a register
`ce`	clock enable: a register loads only on clock edges where ce = 1
CPI	cycles per instruction
DSP48E1	the Artix-7 multiply-accumulate slice; one performs our 8×8 multiply
int8	an 8-bit signed integer, -128 to 127
LUTRAM	small memories built from look-up tables (distributed RAM), read combinationally
MAC	multiply-accumulate: ACC $\leftarrow$ ACC + $a \cdot b$
$n_{1/2}$	half-performance length: the stream length at which half the peak rate is reached
one-hot	a state encoding with one flip-flop per state
ReLU	rectified linear unit: negative values become 0
RQ	requantise: arithmetic shift of the 24-bit sum, optional ReLU, saturation to int8
RTN	register-transfer notation, e.g. P0 $\leftarrow$ P0 + 1
sext_{24}	sign extension to 24 bits: copy the top bit into every new bit
STREAM	the FSM state that runs one element of `MAC #n` per clock
`V1`, `V2`	valid bits of the two pipeline steps after addressing
WNS	worst negative slack: zero or more means the design meets its clock period
XSim, nvc	Vivado's VHDL simulator; a free VHDL simulator for laptops